

06 API - Identity - Security deel 1

📅 09-09-2026 ⌚ Ongeveer 23 minuten 📄 Ongeveer 3434 woorden

Je kunt aan een .NET-applicatie een beveiligingslaag toevoegen met behulp van een membershipsysteem van EntityFramework: Identity. Dit is **een membership systeem** voorzien om een nieuwe gebruikers te registreren, in te loggen, gebruikers te beheren, specifieke gebruikersrechten beheren...

Via een migratie kun je gebruikers- en rollentabellen aanmaken in de database. Daarna maak je gebruik van de Identity-services UserManager, RoleManager en SignInManager om gebruikers en hun rollen te beheren.

Implementatie

Er zijn twee manieren om Identity te voorzien:

- Tijdens het aanmaken van je project
- Toevoegen aan een bestaand project.

In de cursus wordt enkel optie 2 uitgewerkt.

Optie 1: Aanmaken project met Identity

Let op: Kies bij Authentication Type niet voor None maar:



Optie 2: Bestaand project

Om Identity aan een bestaand project toe te voegen, volg je dit stappenplan:

1. Extra EF Core packages installeren
2. DbContext uitbreiden
3. Program.cs uitbreiden
4. Database updaten

1 Extra EF Packages

NuGet Package Manager

- Microsoft.AspNetCore.Identity.EntityFrameworkCore (bevat de core functionaliteiten voor Identity)
- Microsoft.AspNetCore.Diagnostics.EntityFrameworkCore (bevat functionaliteiten om errors tijdens migraties op te vangen)

Terminal

- dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 8.x
- dotnet add package Microsoft.AspNetCore.Diagnostics.EntityFrameworkCore --version 8.x

2 DbContext

In plaats van over te erven van DbContext gebruik je nu **IdentityDbContext** met het type **IdentityUser**.

Pas de DatabaseContext aan:

```
1 public class WebAPIDemoContext: IdentityDbContext<IdentityUser> cs
```

IdentityUser bevat standaardproperties zoals UserName, Password, Email en PhoneNumber... Wil je graag extra properties toevoegen zoals naam, voornaam, geboortedatum, adres... maak je een nieuwe klasse **CustomUser**. Deze klasse erft over van IdentityUser en voegt de extra properties toe. In je DbContext moet je IdentityUser dan vervangen door Customuser.

Maak de klasse CustomUser aan in de map Models:

```
1 public class CustomUser : IdentityUser
2 {
3     [PersonalData]
4     public string? Naam { get; set; }
5     [PersonalData]
6     public DateTime? Geboortedatum { get; set; }
7 }
```

CS

Pas de DbContext aan:

```
1 public class WebAPIDemoContext : IdentityDbContext<CustomUser> {}
```

CS

Het is de bedoeling dat je zo je eigen CustomUser-model gaat gebruiken, omdat je hier attributen kunt toevoegen die niet standaard voorzien zijn door Identity. Een ander voordeel is dat je GDPR kan toepassen.

Opmerking: Als je rekening moet houden met de GDPR-wetgeving, wat vaak het geval zal zijn, dan moet je [PersonalData] voorzien als tag. Voor de GDPR is het ook interessant om de properties nullable te maken.

De CustomUser is geen standaard model en dient dus altijd in de map Models te worden aangemaakt.

In de DbContext controleer je of in de methode OnModelCreating de base methode aangeroepen wordt (als je dat nog niet hebt)

```
1 protected override void OnModelCreating(ModelBuilder modelBuilder)
2 {
3     base.OnModelCreating(modelBuilder); // Dit is belangrijk!
4 }
```

CS

3 Program.cs

Registreer Identity als een service en geef het model CustomUser mee. Registreer ook welke context Identity gebruikt. **IdentityRole()** heb je nodig om in H7 rollen te kunnen gebruiken.

Voeg volgende registratie toe aan Program.cs:

```
1 builder.Services.AddIdentity<CustomUser, IdentityRole>  
   ().AddEntityFrameworkStores<WebAPIDemoContext>();
```

CS

Authorisatie (Welke rechten je hebt binnen de app) is standaard al gekoppeld aan de app. Authenticatie (of je kunt aanmelden of niet) echter nog niet.

Voeg `app.UseAuthentication();` toe aan `Program.cs`.

```
1 app.UseRouting();  
2 app.UseAuthentication();  
3 app.UseAuthorization();
```

CS

Eventueel kan je ook nog extra settings toevoegen, die een effect hebben op de policy van Identity. Denk aan de vereisten voor paswoorden of eventuele brute-force lockout beveiliging. Dit is optioneel maar hou er rekening mee dat je in de praktijk alle mogelijke beveiligingsmaatregelen correctmatig moet instellen, om elke kans op hacking tot een minimum te herleiden.

```
1 builder.Services.Configure<IdentityOptions>(options => {  
2     // Password settings.  
3     options.Password.RequireDigit = true;  
4     options.Password.RequireLowercase = true;  
5     options.Password.RequireNonAlphanumeric = true;  
6     options.Password.RequireUppercase = true;  
7     options.Password.RequiredLength = 6;  
8     options.Password.RequiredUniqueChars = 1;  
9  
10    // Lockout settings.  
11    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);  
12    options.Lockout.MaxFailedAccessAttempts = 5;  
13    options.Lockout.AllowedForNewUsers = true;  
14  
15    // User settings.  
16    options.User.AllowedUserNameCharacters =  
17    "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-._@+";  
18    options.User.RequireUniqueEmail = false;  
    });
```

CS

Tip cybersecurity: Het onderstaande voorkomt dat bots of hackers langdurig geautomatiseerd aanvallen kunnen uitvoeren op het loginsysteem.

1

```
options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);
```

CS

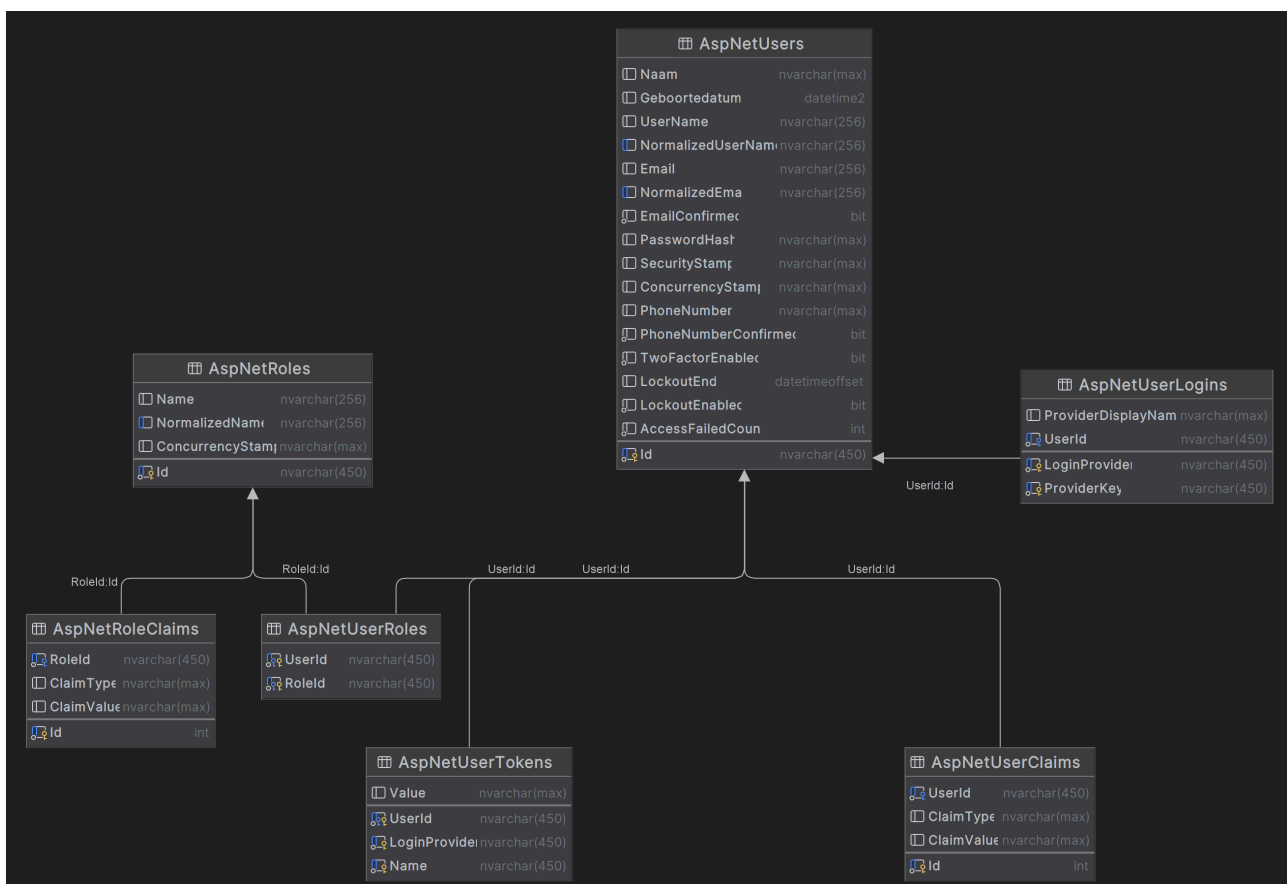
4. Database updaten

Identity maakt gebruik van een aantal tabellen voor gebruikers en rollen. Deze tabellen kun je, na het uitvoeren van de vorige stappen, migreren naar je database:

- Start Nu-Get Package Manager Console:
- Add-Migration AddIdentitySchema
- Update-Database

Spreek voor migraties, in team, altijd duidelijke namen af die effectief afgestemd zijn binnen de groep. (Bijvoorbeeld bij MVC Project)

Nieuwe tabellen in databank



AspNetUsers: Deze tabel bevat alle data van de mogelijke gebruikers, de attributen worden default voorzien door het Identity Framework. Maar als je een CustomUser voorziet, verschijnen deze attributen ook mee in deze tabel!

Een id van een gebruiker ziet er bijvoorbeeld zo uit: c56a4180-65aa-42ec-a945-5fd21dec0538. De primaire sleutel is dus een string, en wordt, bij registratie, ook automatisch als een unieke GUID gegenereerd.

Een aantal attributen: Id (string), UserName, Email en PasswordHash (De gehashte versie van het wachtwoord).

AspNetRoles: Deze tabel bevat de Rollen binnen Identity. Voorbeelden zijn: Gebruiker of Admin. Dit wordt verder besproken in hoofdstuk 7.

AspNetUserRoles: Deze tabel bevat welke rol(len) zijn toegewezen aan welke gebruiker(s). Dit is een associatietabel. Dit wordt ook verder besproken in hoofdstuk 7.

Het Identity Framework kan op twee manieren authorisatie aftoetsen, ofwel Role-Based Security ofwel User-Based Security. Dit hoofdstuk gaat enkel over User-Based Security.

AspNetUserClaims: Om je User-Based Security te implementeren moet je Claims gaan toevoegen bij de inlogprocedure. Daarvoor heb je deze tabel nodig, die reageert op bepaalde instructies om claims aan te geven.

AspNetRoleClaims: Dit wordt verder besproken in hoofdstuk 7.

AspNetUserLogins: Deze tabel kan externe logins bijhouden, een persoon kan bijvoorbeeld inloggen via zijn Google- of Facebook-account.

AspNetUserTokens: In deze tabel worden de tokens bewaard die gekoppeld zijn aan een gebruiker voor bv. een wachtwoord vergeten mogelijkheid of twee stappenverificatie.

Beveiligen van de applicatie

Hieronder worden enkele voorbeelden uitgewerkt voor het beveiligen van de applicatie. Je kunt per ActionResult van een Controller bepalen of dit endpoint toegankelijk is voor een gebruiker of niet. Je gebruikt hiervoor het **[Authorize]-kenmerk**.

BestellingController

Authorize kenmerk

Door dit kenmerk toe te voegen, bepaal je of een Controller of een methode kan aangeroepen worden door een aangemelde gebruiker of niet. Als je **[Authorize]** toevoegt, heb je ook de deze verwijzing nodig: `using Microsoft.AspNetCore.Authorization`. Zet deze best

in GlobalUsings.

Op controller niveau worden alle methodes beveiligd, op voorwaarde dat dit boven de controller staat. Dit wil zeggen dat je om eender welke methode uit deze controller uit te voeren, **alleen maar aangemeld moet zijn, ongeacht welke rol je hebt**. Hierover vind je meer uitleg in H7.

```
1  [Authorize] // alle aangemelde gebruikers hebben toegang tot deze controller cs
2
3  [Route("api/[controller]")]
4  [ApiController]
5  public class BestellingController : ControllerBase
6  {
7      private readonly IUnitOfWork _uow;
8  }
```

Op action method niveau wordt **enkel deze specifieke methode** beveiligd.

```
1  [Authorize] // Alle aangemelde gebruikers hebben toegang tot deze methodes
2  [HttpPost]
3  public async Task<ActionResult<Bestelling>> PostBestelling(Bestelling
4  bestelling)
5  {
6      _context.Bestelling.Add(bestelling);
7      await _context.SaveChangesAsync();
8
9      return CreatedAtAction("GetBestelling", new { id =
bestelling.BestellingID }, bestelling);
10 }
```

Wanneer een methode voorzien is van **Authorize**, kun je ook een authenticatieschema instellen. Dit bepaalt wat er moet gebeuren / getoond worden als je een methode aanspreekt maar niet aangemeld bent of onvoldoende rechten hebt. Later in dit hoofdstuk leer je hoe je een JWTbearer-token kunt gebruiken voor authenticatie. Je zult dan leren hoe je een authenticatieschema instelt. Voorlopig krijg je nu nog een 404 Not Found terug.

AllowAnonymous kenmerk

Het **[AllowAnonymous]**-kenmerk geeft aan dat je voor deze controller of methode **niet** moet ingelogd zijn.

- Op controller niveau zijn alle methodes onbeveiligd

```
1 [AllowAnonymous]
2 [Route("api/[controller]")]
3 [ApiController]
4 public class BestellingController : ControllerBase
```

CS

- Op action method niveau is enkel deze methode onbeveiligd

```
1 [AllowAnonymous]
2 [Route("Lijst")]
3 [HttpGet]
4 public async Task<ActionResult<IEnumerable<Bestelling>>> GetBestelling()
5 {
6     return await _context.Bestelling.ToListAsync();
7 }
```

CS

Combinatie

Belangrijke opmerking is dat je **[Authorize]** en **[AllowAnonymous]** kan gaan combineren (controller + action).

- Controller waar je voor de meeste methodes ingelogd moet zijn, maar voor één of enkele methodes niet:
 - **[Authorize]** op Controller niveau
- Je moet ingelogd zijn
 - **[AllowAnonymous]** op Method niveau Je moet niet ingelogd zijn

In onderstaande voorbeeld, is enkel de methode GetBestelling (route = Lijst) publiek openbaar raadpleegbaar binnen de API.

```
1 [Authorize]
2 [Route("api/[controller]")]
3 [ApiController]
4 public class BestellingsController : ControllerBase
5 {
6     private readonly WebAPIDemoContext _context;
7
8     public BestellingsController(WebAPIDemoContext context)
9     {
10         _context = context;
```

CS

```
11     }
12
13     [AllowAnonymous]
14     [Route("Lijst")]
15     [HttpGet]
16     public async Task<ActionResult<IEnumerable<Bestelling>>>
17     GetBestelling()
18     {
19         return await _context.Bestelling.ToListAsync();
20     }
21
22     [HttpGet("{id}")]
23     public async Task<ActionResult<Bestelling>> GetBestelling(int id)
24     {
25         var bestelling = await _context.Bestelling.FindAsync(id);
26         if (bestelling == null)
27             return NotFound();
28
29         return bestelling;
30     }
31 }
```

Seeding users accounts naar de database

Nu de databasetabellen zijn aangemaakt, ontbreekt er nog een stukje uit de puzzel: **data**.

Je kunt met een sql-script of handmatig in de tabellen gegevens gaan invoeren. Dit is voor gebruikers niet echt een optie omdat de gebruikersId een unieke sleutel moet zijn die niet via Autonummering mag worden bepaald. Zelfregistratie is meestal een goede optie. Hierbij kan een nieuwe gebruiker zelf een account aanmaken. Het probleem hierbij is dat je moet bepalen welke rol een gebruiker krijgt. Hierbij is de aanwezigheid van minstens één beheerder vereist. Je kunt hiervoor bv. de eerste persoon die zich registreert automatisch beheerder maken. Je gaat ook steeds regels voorzien dat de laatste beheerder zichzelf niet kan verwijderen. Je kunt ook regels bepalen dat, na de eerste registratie, de mogelijkheid verdwijnt om zichzelf te registreren. Enkel de beheerder kan dan nog gebruikers aanmaken en beheren. In moderne webapplicaties kun je als gebruiker steeds een zelfregistratieformulier gebruiken om een account aan te maken en te beheren.

Een andere manier is om data te gaan seeden in de database. Dit kan op verschillende manieren, **maar voor Identity** loop je al snel tegen bepaalde beperkingen aan. De reden hiervoor is dat Identity specifieke services gebruikt zoals de UserManager.

Het toevoegen van een IdentitySeeder is echter niet zonder risico. Je gaat op die manier een gebruiker hard-coden waardoor gebruikersnaam en wachtwoord vatbaar zijn voor diefstal en misbruik. Belangrijk is ook dat je dan opvangt dat bij eerste aanmelding van de admin gevraagd wordt om het wachtwoord te wijzigen!

IdentitySeeder.cs klasse

Je voorziet een klasse die de database voor Identity gaat seeden met een try-catch voor eventuele database crashes. Je injecteert deze klasse later als service in Program.cs. Je gaat gebruik maken van de inkomende parameter userManager.

In de Try Catch controleer je of de gebruiker bestaat, zo niet maak je een Admin account aan met het wachtwoord t0LTHxzy.v aan.

```
1      public class IdentitySeeding                                CS
2      {
3          public async Task IdentitySeedingAsync(UserManager<CustomUser>
4      userManager) {
5
6          try
7          {
8              // Gebruiker aanmaken
9              // Admin bestaat nog niet?
10             if (userManager.FindByNameAsync("Admin").Result == null)
11             {
12                 // Gebruiker voorzien
13                 var defaultUser = new CustomUser
14                 {
15                     UserName = "Admin",
16                     Email = "superteam@thomasmore.be",
17                     EmailConfirmed = true
18                 };
19
20                 // Gebruiker aanmaken
21                 await userManager.CreateAsync(defaultUser, "t0LTHxzy.v");
22             }
23         }
24     }
25     catch (DbException ex)
26
27
```

```
28         {
29             throw new Exception(ex.Message.ToString());
30         }
31     }
32 }
```

Maak een service aan in `Program.cs` .

```
1     builder.Services.AddTransient<IdentitySeeding>();
2
3     var app = builder.Build();
```

CS

Voor je `app.Run()` uitvoert, kun je de Seeder uitvoeren. Omdat deze service de UserManager nodig, moet je deze **binnen de scope** van het programma ophalen. Hiervoor gebruiken je de scope en de **ServiceProvider**.

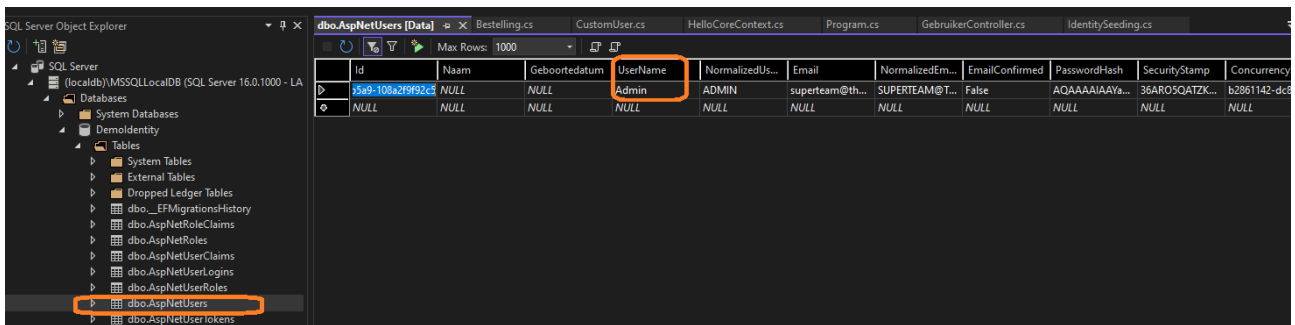
Debugging schakel je hieronder uit, omdat de Seeding Methode al een try-catch implementatie heeft bij failen van database operaties. Als je hier de commentaarlijnen zou weghalen, dan krijg je niet meer de try-catch informatie van de seedingmethode / klasse bij een crash. Hierdoor kan je lang zoeken achter problemen. Vermijd dus best tunnelvisie.

```
1     using (var scope = app.Services.CreateScope())
2     {
3         var seeder =
4         scope.ServiceProvider.GetRequiredService<IdentitySeeding>();
5         UserManager<CustomUser> userManager =
6         scope.ServiceProvider.GetRequiredService<UserManager<CustomUser>>();
7         await seeder.IdentitySeedingAsync(userManager);
8     }
9
10    app.Run();
```

CS

Controleren

Gebruiker in tabel



Id	Naam	Geboortedatum	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash	SecurityStamp	Concurrency
5a9-108a2f992c...	NULL	NULL	Admin	ADMIN	superteam@th...	SUPERTTEAM@T...	False	AQAAAAIAAva...	36AR05QATZK...	b2861142-dc8
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Identity GebruikerController

In de gebruikercontroller ga je zorgen voor Registreren, Inloggen, Lezen van lijsten en details implementeren. Het registreren is bijzonder overeenkomstig met het seeden zoals recent gezien en omvat dezelfde logica.

Je bekijkt even kort de DI's die je gaat gebruiken. Op dit moment volstaat enkel de `_userManager` nog voor het registreren.

```
1 [Route("api/[controller]")]
2 [ApiController]
3 public class GebruikerController : ControllerBase
4 {
5     private UserManager<CustomUser> _userManager;
6     // ..
7     //private RoleManager<IdentityRole> _roleManager;
8
9     public GebruikerController(UserManager<CustomUser> userManager)
10    {
11        _userManager = userManager;
12        // ..
13        //_roleManager = roleManager;
14    }
15 }
```

Registeren

Tip: De informatie over aanmaken van de gebruiker is bijna zo goed als volledig overeenkomstig met de seeding. Maak een **RegistratieDto** in de map **DTO** die de requestelementen van de api body gaat opvangen.

```
1 public class GebruikerRegistratieDto CS
2 {
3     [Required(ErrorMessage = "Naam is verplicht!")]
4     [StringLength(100)]
5     public string Name { get; set; } = "";
6
7     [EmailAddress(ErrorMessage = "Ongeldig emailadres")]
8     [Required(ErrorMessage = "Email is verplicht!")]
9     public string Email { get; set; } = "";
10
11     [Required(ErrorMessage = "Password is verplicht!")]
12     public string Password { get; set; } = "";
13
14     [Required(ErrorMessage = "Tweede wachtwoord is verplicht in te vullen.")]
15     [Compare("Password", ErrorMessage = "De wachtwoorden komen niet overeen.")]
16     public string ConfirmPassword { get; set; } = "";
17
18     public string PhoneNumber { get; set; } = "";
19 }
20 }
```

In de onderstaande methode valideer je eerst of aan de validatieregels van de **RegistratieDto** voldaan zijn. Daarna zoek je of de gebruiker reeds bestaat of niet. Daarna ga je deze gebruiker aanmaken en een rol toekennen. Als dit niet lukt ga je de errors opvangen geef je een BadRequest statuscode terug.

```
1 // Registeren als nieuwe gebruiker CS
2 [HttpPost("register"), AllowAnonymous]
3 public async Task<IActionResult> Register(GebruikerRegistratieDto
4 request)
5 {
6     if (!ModelState.IsValid)
```

```
7         return BadRequest(ModelState);
8
9     var gebruiker = await _userManager.FindByEmailAsync(request.Email);
10    if (gebruiker == null)
11    {
12        var user = new CustomUser
13        {
14            UserName = request.Email,
15            NormalizedUserName = request.Email,
16            Email = request.Email,
17            PhoneNumber = request.PhoneNumber,
18            EmailConfirmed = true,
19            PhoneNumberConfirmed = true
20        };
21        var result = await _userManager.CreateAsync(user,
22    request.Password);
23        //await _userManager.AddToRoleAsync(user, request.Rol);
24
25        if (result.Succeeded)
26            return Ok();
27        else
28        {
29            if (result.Errors.Count() > 0)
30            {
31                foreach (var error in result.Errors)
32                    ModelState.AddModelError("message",
33    error.Description);
34            }
35            return BadRequest(ModelState);
36        }
37    }
38    else
39    {
40        ModelState.AddModelError("message", "Gebruiker is aanwezig in
    database.");
        return BadRequest(ModelState);
    }
}
```

Token

Het inloggen is een procedure die veel elementen bevat. De technieken zijn vaak complex. Het principe is als volgt: Je meldt je aan met je gebruikersnaam en wachtwoord. Dat stuk is nog redelijk eenvoudig. Daarna wordt een JWT-token (JsonWebToken) aangemaakt en teruggegeven aan de gebruiker. Dat is een behoorlijk complexe setup. De gebruiker moet daarna dit token toevoegen aan zijn http-request om te bewijzen dat hij rechten heeft op dat Endpoint.

Om het JWT-token te maken, voeg je in `appsettings.json` onderstaande gegevens toe. De secretsleutel wordt gebruikt om het 'token' te vormen. De andere waardes dienen als extra validatie.

```
1  "MySettings": {
2    "Secret": "d4f.5E6a7-8b9c-0d1e-WfGl1m-4h5i6j7k8l9m",
3    "ValidAudiences": [ "https://localhost:44359",
4    "https://localhost:49153" ],
5    "ValidAudience": "https://localhost:44359",
6    "ValidIssuer": "https://localhost:44359"
  }
```

Om deze gegevens gestructureerd te kunnen ophalen, heb je een model `MySettings.cs` nodig.

```
1  public class MySettings
2  {
3      public string? Name { get; set; }
4      public char[]? Secret { get; set; }
5      public string? ValidIssuer { get; set; }
6      public string? ValidAudience { get; set; }
7      public List<String>? ValidAudiences { get; set; }
8  }
```

In een nieuwe map `Helper` komt de statische `Token` -klasse om het token uiteindelijk aan te maken op basis van de gegevens in `appsettings.json`.

```

1  public static class Token
2  {
3
4      public static MySettings? mySettings;
5
6      public static JwtSecurityToken GetToken(List<Claim> authClaims)
7      {
8          var authSigningKey = new
9      SymmetricSecurityKey(Encoding.ASCII.GetBytes(mySettings!.Secret!));
10         var token = new JwtSecurityToken(
11             issuer: mySettings!.ValidIssuer!,
12             audience: mySettings!.ValidAudience!,
13             expires: DateTime.Now.AddHours(3),
14             claims: authClaims,
15             signingCredentials: new SigningCredentials(authSigningKey,
16 SecurityAlgorithms.HmacSha256));
17         return token;
18     }
19 }

```

In Program.cs haal je de gegevens op uit appsettings.json en plaatst deze in het MySettings-model. Dit model geef je door naar de statische Token generatie klasse. Deze waarden worden standaard opgehaald uit de json-file. Indien er een waarde ontbreekt, treden de dubbele vraagtekens in werking. Je wil ten alle tijde vermijden dat het programma crasht, maar ook problemen in de statische klasse optreden.

```

1  var builder = WebApplication.CreateBuilder(args);
2
3  // We gebruiken de MySettings in Token Class, de Login methode heeft deze n
4  // Token overpompen in MySettings
5  Token.mySettings = new MySettings
6  {
7      Secret = (builder.Configuration["MySettings:Secret"] ?? "d4f.5E6a7-8b9c
8  WfG11m-4h5i6j7k8l9m").ToCharArray(),
9      ValidIssuer = builder.Configuration["MySettings:ValidIssuer"] ??
10 "https://localhost:7055",
11      ValidAudience = builder.Configuration["MySettings:ValidAudience"] ??
12 "https://localhost:7055"
13 };
14
15 builder.Configuration.GetRequiredSection(nameof(MySettings)).Bind(Token.myS

```

Packages

Bij het inloggen wordt het JWT-token (JSON Web Token) aangemaakt. Om dit mee te geven in Swagger als authenticatie, heb je volgende package nodig:

Visual Studio

```
1      <PackageReference                                CS
      Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.8"
      />
```

Terminal

```
1      dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer -- psw
      version 8.0.8
```

De term JWT Bearer betekent dat het JWT-token wordt gebruikt als een Bearer-token in een authenticatiesysteem. Dit betekent dat het token zelf (een JWT) wordt meegestuurd in een HTTP-header, en de server vertrouwt erop dat de houder (bearer) van het token geautoriseerd is.

In `Program.cs` moet het JWTBearer gekoppeld worden aan onze API voor de **authenticatie** van de gebruiker.

Bij `AddJwtBearer` zijn een aantal waardes uitgeschakeld door de docenten, omdat deze als bugfixes in 8.0 worden aangehaald. Een interessante bron die je hierbij kan helpen: <https://github.com/dotnet/aspnetcore/issues/52286>

```
1      var audiences = Token.mySettings.ValidAudiences;                                CS
2
3      // Authentication toevoegen
4      builder.Services.AddAuthentication(options =>
5      {
6          options.DefaultAuthenticateScheme =
7          JwtBearerDefaults.AuthenticationScheme;
8          options.DefaultChallengeScheme =
          JwtBearerDefaults.AuthenticationScheme;
```

```
9         options.DefaultScheme = JwtBearerDefaults.AuthenticationScheme;
10     })
11     // Jwt Bearer toevoegen
12     .AddJwtBearer(options =>
13     {
14         options.IncludeErrorDetails = true; // debugging
15         options.SaveToken = true;
16         options.RequireHttpsMetadata = false;
17         options.UseSecurityTokenValidators = true; // fix bug 8.0
18         options.TokenValidationParameters = new TokenValidationParameters()
19         {
20             ValidateLifetime = false, // onderdrukken bug 8.0
21             //ValidateIssuer = true,
22             //ValidateAudience = true,
23             ValidateIssuer = false,
24             ValidateAudience = false,
25             //ValidAudiences = audiences,
26             //ValidIssuer = Token.mySettings.ValidIssuer,
27             IssuerSigningKey = new
28 SymmetricSecurityKey(Encoding.ASCII.GetBytes(Token.mySettings.Secret))
29         };
30         // Customize the JWT Bearer events
31         options.Events = new JwtBearerEvents
32         {
33             OnChallenge = context =>
34             {
35                 context.HandleResponse();
36                 // Customize unauthorized response
37                 context.Response.StatusCode = 401; // Unauthorized
38                 context.Response.ContentType = "application/json";
39                 var result = "{\"error\": \"Niet geauthenticeerd. Token is
40 ongeldig of ontbreekt.\"}";
41                 return context.Response.WriteAsync(result);
42             },
43             OnForbidden = context =>
44             {
45                 // Customize forbidden response
46                 context.Response.StatusCode = 403; // Forbidden
47                 context.Response.ContentType = "application/json";
48                 var result = "{\"error\": \"Toegang geweigerd. U heeft niet
49 de juiste rechten voor deze actie.\"}";
50                 return context.Response.WriteAsync(result);
51             }
52         }
53     }
54 }
```

```
    }  
    }  
});  
  
builder.Services.AddEndpointsApiExplorer();
```

Swagger

Breid Swagger uit zodat je een inlogscherf krijgt waar je het Token kunt plakken.

```

1 builder.Services.AddSwaggerGen(swagger => {
2     //This is to generate the Default UI of Swagger Documentation
3     swagger.SwaggerDoc("v1", new OpenApiInfo
4     {
5         Version = "v1",
6         Title = "AuthTest Web API",
7         Description = "Authentication and Authorization in ASP.NET with
8 JWT and Swagger"
9     });
10    // To Enable authorization using Swagger (JWT)
11    swagger.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme()
12    {
13        Name = "Authorization",
14        //Type = SecuritySchemeType.ApiKey,
15        Type = SecuritySchemeType.Http,
16        Scheme = "Bearer",
17        BearerFormat = "JWT",
18        In = ParameterLocation.Header,
19        Description = "Enter ZONDER Bearer. Dus gewoon je TOKEN:
20 \r\n\r\nExample: \"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9\"\"",
21    });
22    swagger.AddSecurityRequirement(new OpenApiSecurityRequirement
23    {
24        {
25            new OpenApiSecurityScheme
26            {
27                Reference = new OpenApiReference
28                {
29                    Type = ReferenceType.SecurityScheme,
30                    Id = "Bearer"
31                }
32            }
33        }
34    });
35 }
36 }

```

```
33         },
34         new string[] {}
    }
    });
});
```

Inloggen

Maak een `GebruikerInlogDto` aan om enkel de gebruikersnaam en wachtwoord op te vragen:

```
1  public class GebruikerLoginDto
2  {
3      [EmailAddress(ErrorMessage = "Ongeldig emailadres")]
4      [Required(ErrorMessage = "Email is verplicht!")]
5      public string Email { get; set; } = "";
6
7      [Required(ErrorMessage = "Password is verplicht!")]
8      [DataType(DataType.Password)]
9      public string Password { get; set; } = "";
10 }
```

Zorg dat je de `_signInManager` kan gebruiken. Gebruik Dependency Injection om deze beschikbaar te stellen voor de controller.

In onderstaande code ga je **eerst eventuele problemen detecteren**, zoals een nog niet geactiveerd account, een verkeerde loginpoging of een geblokkeerd account.

Als er geen problemen zijn vastgesteld, ga je de claims tabel en rol opvullen, je gebruikt ook een security token. Verwacht niet dat je onderstaande syntax volledig begrijpt, maar het is noodzakelijk om de inlogprocedure op te starten. Voor meer informatie kan je terecht bij je docent.

Hoe een token precies werkt met cryptografische technieken, behoort tot een universitaire opleiding en valt buiten beschouwing van deze cursus. Deze statische klasse wordt reeds voorzien door de docenten. In simpele klare taal helpt een token als security maatregel een extra **unieke sleutel** toe te voegen.

```
1 [HttpPost("login")]
2 [AllowAnonymous]
3 public async Task<IActionResult> Login(GebruikerLoginDto model)
4 {
5     if (!ModelState.IsValid)
6         return BadRequest(ModelState);
7
8     var user = await _userManager.FindByEmailAsync(model.Email);
9     if (user != null && !user.EmailConfirmed)
10    {
11        ModelState.AddModelError("message", "Emailadres is nog niet
12bevestigd.");
13        return BadRequest(model);
14    }
15    if (await _userManager.CheckPasswordAsync(user, model.Password) ==
16false)
17    {
18        // Nooit exacte informatie geven: zeg alleen dat combinatie
19vekeerd is...
20        ModelState.AddModelError("message", "Verkeerde
21logincombinatie!");
22        return BadRequest(model);
23    }
24
25    var result = await _signInManager.PasswordSignInAsync(user.UserName,
26model.Password, false, true);
27
28    if (result.IsLockedOut)
29        ModelState.AddModelError("message", "Account geblokkeerd!!");
30
31    if (result.Succeeded)
32    {
33        var authClaims = new List<Claim>
34        {
35            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
36            new Claim(ClaimTypes.Name, user.UserName),
37            new Claim(JwtRegisteredClaimNames.Jti,
38Guid.NewGuid().ToString()),
39        };
40
41        var userRoles = await _userManager.GetRolesAsync(user);
42        if (userRoles != null)
43        {

```

CS

```
43         foreach (var userRole in userRoles)
44             authClaims.Add(new Claim(ClaimTypes.Role, userRole));
45     }
46
47     var token = Token.GetToken(authClaims);
48
49     return Ok(new
50     {
51         token = new JwtSecurityTokenHandler().WriteToken(token),
52         expiration = token.ValidTo
53     });
54 }
55
56 ModelState.AddModelError("message", "Ongeldige loginpoging");
57 return Unauthorized(ModelState);
58 }
```

Ophalen gebruikers

Beveilig de controller zodat enkel het registreren en inloggen toegankelijk is voor iedereen. De rest van de controller moet beveiligd zijn. Het ophalen van gebruikers verloopt nadien op een gelijkaardige manier als de vorige hoofdstukken.

```
1 // Lijst gebruikers
2 [Route("Lijst")]
3 [HttpGet]
4 public async Task<ActionResult> GetLijst()
5 {
6     return Ok(await _userManager.Users.ToListAsync());
7 }
8
9 // Eén gebruiker op basis van id
10 [HttpGet("{id}")]
11 public async Task<ActionResult> Details(string id)
12 {
13     CustomUser? gebruiker = await _userManager.Users.Where(u => u.Id ==
14     id).FirstOrDefaultAsync();
15
16
17
18 }
```

```
    if (gebruiker != null)
        return Ok(gebruiker);
    else return Ok(null);
}
```

Uittesten

Psst: Je kan stap 2+3 ook overslaan, omdat bij de seeding een default account wordt aangemaakt. Gebruik die gegevens dan.

1. Start de applicatie
2. Test met SwaggerUI: **Registreren**
3. Maak een nieuwe user.
4. Test met SwaggerUI: **Inloggen** en gebruik de aangemaakte gegevens van daarnet om in te loggen
5. Resultaat:

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'https://localhost:44359/api/gebruiker/login' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "email": "dries.hoefkens@gmail.com",
  "password": "Test123.123"
}'
```

Request URL

```
https://localhost:44359/api/gebruiker/login
```

Server response

CodeDetails

200

Response body

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZWlkeS54bWxzbnZlLnR5c2lwYyZmNmFzLnhtbHNvYXAub3NlL3dzLzIwMDUwMDUwMWRlbnpkdkV2ZxhaW1ZlZ5bldqZmlhcystYW93bnZnOUY2ZlZlLWQvMDVvaWRlbnpkdkV2ZxhaW1ZlZ3VGVuOjE1hjZ6lpblI:ZhgfDmsr9tcl0B-gr0-PwKdxJvRmgUusrBhf14",
  "expiration": "2024-09-11T12:52:10Z"
}
```

Download

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Wed, 11 Sep 2024 09:52:10 GMT
expires: Thu, 01 Jan 1970 00:00:00 GMT
pragma: no-cache
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code	Description	Links
200	OK	No links

6. Nu kan je de token gebruiken, die weergegeven wordt. Simpel: Kopieer deze.
7. Configureer/Plak deze in Auth scherm van Swagger UI. Rechts boven elke API-methode
OF Rechts bovenaan kan je auth scherm oproepen.
8. Als je nu de **Lijst** bekijkt geeft hij de lijst en krijg je geen Bad token.

Hier vind je het WebAPIDemo-project.

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM